

Predicting Backdoor Recovery Cost from ASR-G in Large Vision-Language Models

Tharun Varma Dendukuri

12/5/26

Abstract

Backdoor attacks on large vision-language models (LVLMs) are usually judged by how effective they are and by how well they survive a change in the data distribution, measured by the ASR-G metric. A high ASR-G means the backdoor keeps working even when the deployment data differs from the attacker’s. This tells us how dangerous an attack is, but not what it costs to fix. This paper presents a way to *predict* the **recovery cost** of an LVLM backdoor, which consists of the compute, clean data, and training needed to suppress the attack while keeping the model useful. On BLIP-2, 20 backdoors are planted and 120 recovery experiments are run across three repair strategies and two data budgets, measuring which runs fail to recover in time. Recovery cost rises sharply with ASR-G: about **$2.9\times$ for every $+0.1$ in ASR-G (95% CI $1.4\text{--}6.9\times$)**. Before any repair, it is easier to predict *whether* a model can be recovered than *how much* that recovery will cost. Low cost targeted repair works for low-ASR-G backdoors, but high-ASR-G ones need full retraining, and repair becomes impractical once ASR-G passes about **0.43**. In short, ASR-G can be used not only to describe an attack but to plan its remediation.

1. Introduction

A backdoored model works normally on clean inputs but gives an attacker-chosen output when a hidden trigger appears. Large vision-language models (LVLMs) are composed of third-party parts—vision encoders, language models, instruction data, adapters—so any of these can carry a planted backdoor [10, 13]. Two lines of work have grown around this: one on *attacks* (making triggers stealthy, strong, and robust to distribution shift) and one on *defenses* (removing a backdoor once it is found).

A recent idea evaluates attacks under domain shift. Liang et al. [1] define ASR-G, the attack success rate when the test distribution differs from attacker’s. Some backdoors keep working across this shift and some do not: an attribution-guided attack (MABA) stays strong, while simple patch triggers fade. A high ASR-G therefore marks a backdoor that is not tied to one narrow input style but is woven into features the model uses widely.

Once a backdoor is found, the operator faces a question that neither line of work answers: **how much will it cost to remove?** Retraining an LVLM adapter can take thousands of GPU-hours, and even light fine-tuning uses compute, clean data, and time. The operator must choose whether to repair or replace the model, how much to spend, and which method to use before incurring the cost.

This paper argues that ASR-G, which is already known at detection, predicts how hard a backdoor is to remove. The reason is simple. A high-ASR-G backdoor generalizes because its trigger competes with real content across domains, so it is tangled up with the features the model depends on and resists light fine-tuning. A low-ASR-G backdoor leans on a narrow, domain-specific cue that a targeted fix can cut out cheaply. There is prior evidence that how a backdoor is embedded controls how hard it is to clean: Verma et al. [4] show that for CLIP-style models, a backdoor planted under a stronger pre-training objective cannot be removed no matter how much data or training they used—a sign that some backdoors are simply too costly to fix.

This paper tests the idea in the first study that predicts LVLM backdoor recovery cost from pre-repair features, and the first to measure that cost systematically rather than reporting it in passing. The contributions are: (1) a protocol that plants backdoors across attacks, poisoning rates, and trigger strengths on BLIP-2 and records the compute, data, and steps each takes to remove, keeping—not discarding—the runs that never finish; (2) evidence that ASR-G predicts removal cost, measured with attack fixed effects so the effect comes from variation *within* an attack rather than from which attack it is; (3) a predictor tested by holding out one whole attack at a time, including an adaptive one, with an ablation that isolates ASR-G’s contribution; (4) a comparison of repair strategies by cost as ASR-G grows, plus a simple rule for picking one; and (5) a cost-based grouping of backdoors and evidence for an ASR-G level past which repair is impractical. The goal is defensive.

2. Related Work

Backdoor attacks. Early work used patch and blended triggers [10, 11]. Later triggers became harder to see or adapted to the input, through warping [12] and, in the multimodal setting, dual-key triggers [13]. Liang et al. [1] re-evaluate these under domain shift, define ASR-G, and propose MABA. This work reuses their attack grouping (static / content-aware / model-adaptive) and their ASR-G metric, but where they describe attacks, it measures what it costs to undo them.

Backdoor removal. Many defenses repair a poisoned model with a little clean data: fine-pruning [7], attention distillation [8], adversarial neuron pruning [5], reconstructive neuron pruning [6], and adversarial unlearning [9], with newer inversion-free methods in 2025 [14]. These papers rarely report cost, and when they do it is for one method in one setting, never as a function of the attack. This work uses retraining,

full fine-tuning, and targeted repair as its strategies—the pruning and unlearning methods above fit the same interface—and makes their cost the main question.

When are backdoors hard to remove? The closest work is Verma et al. [4]: how cleanable a VLM backdoor is depends on the pre-training objective, and under a stronger objective the backdoor survives even when clean data is doubled and training runs five times longer. This supports two premises used here—removal cost tracks how deeply the backdoor is embedded, and some backdoors are effectively unfixable. Game-theoretic unlearning [15] studies the attacker’s cost of keeping a backdoor alive, not how attack strength drives removal cost.

Benchmarks. BackdoorVLM [2] and BackdoorLLM [3] standardize how well attacks and defenses *work*. This study is complementary: it adds the cost layer they leave out, together with a model that predicts it.

3. Problem Formulation

Threat and recovery model. An attacker poisons a fraction p of the training data so the model outputs a target string t whenever a trigger is present. A defender finds the backdoor (finding it is out of scope), has some clean in-distribution data, picks a strategy and a budget, and repairs the model. This paper measures what the repair costs.

ASR-G. For a domain d , ASR_d is the share of triggered inputs whose output contains t , counting only inputs whose clean caption does not already contain t . Following [1],

$$\text{ASR-G} = \min\left(1 - \frac{|\text{ASR}_{D_k} - \text{ASR}_{D_t}|}{\max(\text{ASR}_{D_k}, \text{ASR}_{D_t})}, 1\right) \in [0, 1], \quad (1)$$

where D_k is the attacker’s domain and D_t the shifted one the user sees; $\text{ASR-G} = 1$ means the attack fully survives the shift. Since evaluation uses several shifted domains, the main number is the mean attack success over them; the retention ratio (shifted over in-domain) and the worst single domain are also reported as checks.

Recovery cost. A run is checked at regular intervals and called *recovered* the first time

$$\text{ASR}_{\text{in}} \leq \tau \wedge \overline{\text{ASR}}_{\text{shift}} \leq \tau \wedge U_{\text{in}} \geq U_{\text{ref}} - \delta, \quad (2)$$

holds: attack success is below τ both in-domain and across shifted domains, *and* clean utility stays within δ of the model before repair. The utility floor matters—without it, simply breaking the model would count as a cheap fix. Success is confirmed on the final metric (CIDEr), and a faster token-F1 proxy is used only to decide how often to check. Cost is counted in optimization steps (the main measure within one strategy), clean samples used, and GPU-hours; to compare *across* strategies, GPU-seconds are used, since a step costs different amounts for different strategies. A run that hits the step cap without recovering is right-censored: its true cost is at least the cap. A repair is *prohibitive* if the run is censored or costs as much as full retraining.

4. Method

Pre-recovery features (all measurable before any repair). *Attack*: family, poisoning rate, ASR_{in} , and ASR-G . *Trigger*: area, L_2/L_∞ size, how distinctive it is to a clean model, and how much input-gradient saliency falls on it. *Embedding*: how much each block reacts to the trigger beyond a clean model’s reaction, how deep and how spread out that reaction is, the alignment between clean and backdoor gradients, and the target-string loss. *Model*: parameter counts and number of repairable blocks. An optional persistence probe (a short clean fine-tune, with its own cost recorded) is reported both with and without, so the “predict before repair” claim does not rely on it.

Remediation strategies. *Full retraining* starts from the clean pre-attack weights (the “replace” baseline). *Full fine-tuning* updates the whole trainable scope. *Targeted repair* reinitializes and fine-tunes only the blocks the embedding features flag as most affected. All three use the same budgets and the same cost meter. Published defenses—fine-pruning [7], ANP [5], I-BAU [9]—fit the same interface and can be added.

Analysis. Each row is one recovery run; runs are grouped by backdoored model, and all confidence intervals bootstrap over those models. Because many runs are censored, the cost is fit with a log-normal accelerated-failure-time (AFT/Tobit) model, which treats a censored run as a lower bound rather than throwing it away. Each ASR-G effect is reported on its own and again with attack fixed effects, where only within-attack variation identifies it; the fixed-effects number is the one relied on here. The cost predictor (ridge and gradient-boosted trees) is tested by holding out one whole attack at a time, including an adaptive one.

5. Experimental Setup

Model. BLIP-2 (OPT-2.7B). Only the Q-Former, its query tokens, and the language projection are trained; the vision encoder and language model stay frozen. This is a deliberately small, single-model study; running more architectures is future work.

Attacks and grid. Five attacks—BadNets, Blended, WaNet, DualKey, and an EOT-trained, domain-agnostic *proxy* for MABA (the official MABA is not used; the proxy is labeled as such and never called MABA)—each at poisoning rates of 1% and 10% with two trigger strengths and one seed. That gives 20 backdoored models, and 120 recovery runs across three strategies and two budgets (1k and 5k clean samples). Recovery runs for up to 2000 steps, checked every 100 steps, on one NVIDIA A100.

Data and domains. The in-domain task is COCO captioning. The shifted domains that define ASR-G are four real out-of-distribution sets—NoCaps (out-domain), sketch, painting, and cartoon—not synthetic style transfers. Utility is CIDEr, with token-F1 as a quick proxy during a run. Attacker data, the defender’s clean pool, and the evaluation set are kept separate so none leaks, and the budgets are nested. The

thresholds are $\tau = 0.05$ and $\delta = 0.02$, and the learning rates were fixed on a clean control before the main runs.

6. Results

Of the 120 runs, 73 (61%) recovered within the 2000-step cap; the rest were censored. Full retraining recovered every model (it always can, since it starts clean), targeted repair recovered 60%, and full fine-tuning only 22%—plain fine-tuning usually cannot clean these backdoors in the budget. Three low-poison models barely took the backdoor (one Blended and two WaNet, with in-domain ASR ≤ 0.38); they are kept as genuine low-ASR-G points.

RQ1 — Cost rises with ASR-G (supported). With attack fixed effects, the ASR-G coefficient is positive and its 95% CI excludes zero: cost grows $2.9\times$ for every $+0.1$ in ASR-G (95% CI $1.4\text{--}6.9\times$), and the marginal estimate is close at $2.5\times$. The result holds under all three ASR-G definitions (Table 1), and for full fine-tuning the recovery rate drops steadily as ASR-G rises (0.57, 0.08, 0 across tertiles; Fig. 1). This is the main result.

RQ2 — Recoverability is predictable; exact cost less so (partial). Holding out one attack at a time, a classifier predicts whether a model can be recovered in budget with AUC 0.79. Predicting how much it will cost is harder: the best feature set (all pre-recovery features plus the probe) reaches Spearman 0.56 and puts 30% of runs within $2\times$ of the true cost (Table 2). Two results still point to ASR-G: on its own it predicts better than the attack label (log-steps MAE 2.47 vs. 3.13), and dropping it hurts the held-out error (the held-out adaptive attack goes from 3.2 to 5.3). Permutation importance ranks the embedding-depth features above ASR-G, but that is because those features are correlated, not because ASR-G is useless—the ablation is the right test.

RQ3 — Targeted repair wins at low ASR-G (directional). For most low-ASR-G models, targeted repair is the cheapest fix that works—often removing the backdoor with *no* fine-tuning at all (14 of its 24 successes recover right after reinitialization, with utility intact). At middle and high ASR-G, full retraining is the cheapest that still works. The link between the best strategy and the ASR-G tertile points the right way but is not significant here ($\chi^2(4)=7.2$, $p=0.13$). Picking the strategy with the lowest predicted cost cuts the average regret to $2.7\times$, against $4.0\times$ for always retraining.

RQ4 — Proportionality: inconclusive. The cost-vs-ASR-G elasticity is 3.0, but its 95% CI ($-0.2, 4.8$) includes 1, so super-linear growth cannot be claimed. The proxy’s ASR-G values are bunched together (0.69–0.87) and several runs censor, which leaves this test under-powered.

RQ5 — A critical ASR-G level (supported). A two-piece fit beats a straight line by a wide margin ($\Delta\text{BIC} = -15$), with the break at ASR-G ≈ 0.43 (0.43–0.48 across definitions). Full fine-tuning is prohibitive almost everywhere (its 50% point is near ASR-G 0.03), while targeted repair crosses the 50% mark around ASR-G 0.45 (Fig. 3). Past this point, repairing within a fixed budget is out of reach—the same pattern Verma et al. report [4].

Table 1: RQ1. Recovery-cost multiplier per $+0.1$ ASR-G (AFT, non-retrain runs), robust across ASR-G definitions. Attack-FE is the claim-bearing estimate.

ASR-G definition	cost mult./ $+0.1$	Spearman
mean-shifted (primary)	$2.48\times$	0.38
normalized retention	$2.57\times$	0.38
worst-case (min)	$1.96\times$	0.34
mean-shifted, + attack FE	$2.9\times$ (CI 1.4–6.9)	—

Table 2: RQ2. Leave-one-attack-out cost prediction (target: $\log(1+\text{steps})$; constant-baseline MAE 2.93). ASR-G alone beats attack-type; removing it hurts.

features	model	MAE	within $2\times$	Spearman
ASR-G only	ridge	2.47	0.15	0.39
attack-type only	ridge	3.13	0.00	−0.06
pre-recovery	ridge	2.83	0.21	0.58
pre-recovery+probe	ridge	2.58	0.30	0.56
− ASR-G	ridge	3.26	0.18	0.50

7. Discussion

ASR-G is a cost signal. Since cost grows about $2.9\times$ per $+0.1$ in ASR-G—within attack and under every definition—an operator can read off a rough repair cost from a number already in hand at detection. The features show why: the depth of the trigger’s effect on activations and the target-string loss carry most of the signal, which fits the idea that high-ASR-G backdoors sit in the model’s core features.

A cost-based grouping. Below ASR-G ≈ 0.43 , backdoors are cheap to fix—often by resetting a few affected blocks—so repair makes sense. Above it, only full retraining reliably works, and its cost sets the floor, so replacing the model becomes the better call. This gives incident response a concrete number for the repair-vs-replace decision, in line with [4].

What this paper does not show. Whether cost grows faster than linearly (RQ4) is unsettled at this size; whether the pattern holds beyond BLIP-2 is future work.

8. Limitations

This is one model (BLIP-2) with one seed per setting, so the pattern is established here but not shown to generalize, and the intervals—which come from 20 models, not repeated seeds—are preliminary. The strategies are from the fine-tuning, pruning, and unlearning family; other kinds of defense would add to the picture. GPU-hours depend on the hardware, so steps and a single pinned A100 are used instead. The adaptive attack is an EOT proxy rather than official MABA, and its narrow ASR-G range is what leaves RQ4 open. At 1% poison, two WaNet settings and one Blended setting barely took the backdoor; they are kept as real low-ASR-G points.

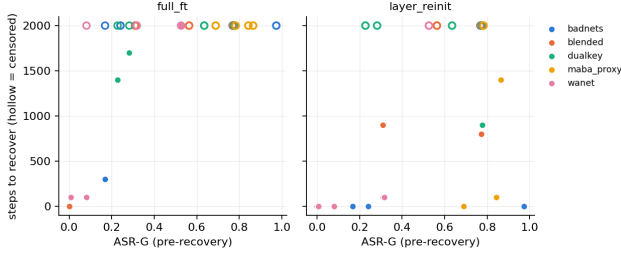


Figure 1: RQ1. Steps to recover vs. ASR-G; hollow markers are censored (hit the step cap). Full fine-tuning (left) censors almost everywhere except at low ASR-G; targeted reinit (right) clears low-ASR-G backdoors at ~ 0 steps and censors as ASR-G rises.

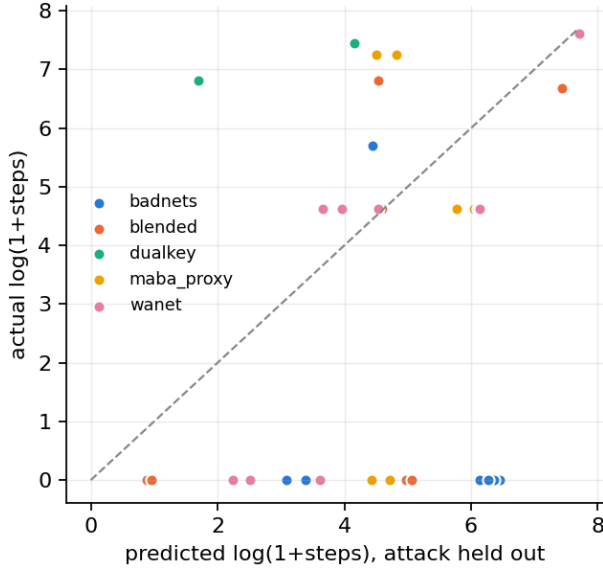


Figure 2: RQ2. Predicted vs. actual $\log(1+\text{steps})$, attack held out.

9. Conclusion

This paper shows that ASR-G, usually read as a measure of how well a backdoor attacks, also predicts how expensive it is to remove. On BLIP-2, cost grows about $2.9 \times$ per $+0.1$ in ASR-G within attack, whether a model is recoverable can be predicted in advance (AUC 0.79), targeted repair is cheapest for low-ASR-G backdoors while high-ASR-G ones need full retraining, and repair becomes impractical past ASR-G ≈ 0.43 . The next steps are more architectures and the official MABA attack.

Ethics and responsible disclosure. This work is about *removing* backdoors. Every attack used is already published, so no new attack is added. The experiments use self-trained models, public captioning data, and a harmless placeholder target. The measurement and analysis code are released, but no poisoned model weights.

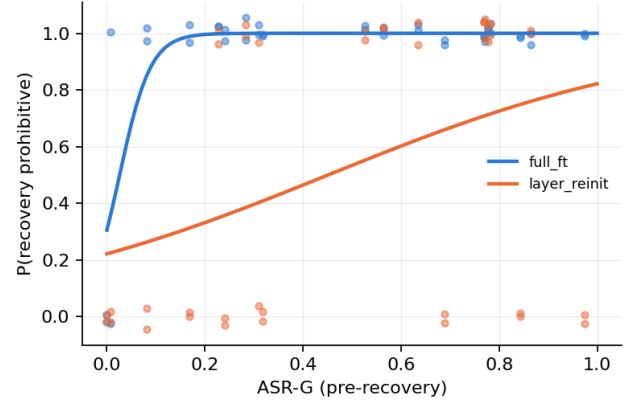


Figure 3: RQ5. $P(\text{recovery prohibitive})$ vs. ASR-G per strategy; the crossing marks the critical level.

Reproducibility. The code, configurations, and full run tables are released at <https://github.com/tdhdap/Predicting-Backdoor-Recovery-Cost-from-ASR-G-in-Large-Vision-Language-Models>, and every number here comes from them.

References

- [1] S. Liang, J. Liang, T. Pang, C. Du, A. Liu, M. Zhu, X. Cao, D. Tao. Revisiting Backdoor Attacks against Large Vision-Language Models from Domain Shift. *CVPR*, 2025. arXiv:2406.18844.
- [2] BackdoorVLM: A Benchmark for Backdoor Attacks on Vision-Language Models. 2025. arXiv:2511.18921.
- [3] Y. Li, H. Huang, et al. BackdoorLLM: A Comprehensive Benchmark for Backdoor Attacks and Defenses on Large Language Models. *NeurIPS*, 2025. arXiv:2408.12798.
- [4] A. Verma, et al. Effective Backdoor Mitigation in Vision-Language Models Depends on the Pre-training Objective. *TMLR*, 2025. arXiv:2311.14948.
- [5] D. Wu, Y. Wang. Adversarial Neuron Pruning Purifies Backdoored Deep Models. *NeurIPS*, 2021.
- [6] Y. Li, X. Lyu, N. Koren, L. Lyu, B. Li, X. Ma. Reconstructive Neuron Pruning for Backdoor Defense. *ICML*, 2023.
- [7] K. Liu, B. Dolan-Gavitt, S. Garg. Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks. *RAID*, 2018. arXiv:1805.12185.
- [8] Y. Li, X. Lyu, N. Koren, L. Lyu, B. Li, X. Ma. Neural Attention Distillation: Erasing Backdoor Triggers from Deep Neural Networks. *ICLR*, 2021. arXiv:2101.05930.
- [9] Y. Zeng, S. Chen, W. Park, Z. M. Mao, M. Jin, R. Jia. Adversarial Unlearning of Backdoors via Implicit Hypergradient. *ICLR*, 2022. arXiv:2110.03735.
- [10] T. Gu, B. Dolan-Gavitt, S. Garg. BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain. arXiv:1708.06733, 2017.
- [11] X. Chen, C. Liu, B. Li, K. Lu, D. Song. Targeted Backdoor Attacks on Deep Learning Systems Using Data Poisoning. arXiv:1712.05526, 2017.

- [12] A. Nguyen, A. Tran. WaNet – Imperceptible Warping-based Backdoor Attack. *ICLR*, 2021. arXiv:2102.10369.
- [13] M. Walmer, K. Sikka, I. Sur, A. Shrivastava, S. Jha. Dual-Key Multimodal Backdoors for Visual Question Answering. *CVPR*, 2022. arXiv:2112.07668.
- [14] Y. Chen, et al. REFINE: Inversion-Free Backdoor Defense via Model Reprogramming. *ICLR*, 2025. arXiv:2502.18508.
- [15] A Game-Theoretic Framework for Strategic Machine Unlearning in Backdoor Mitigation. 2026.